

ORDERS AND ORDER DETAILS MANAGEMENT SYSTEM

Introduction:

An e-commerce platform manages customer orders and the products included in each order. The **Orders and Order Details Management System** is designed to store overall order information and individual product details.

The system helps to maintain customer orders, connect orders with products, and store information such as order date, total amount, quantity and price. It also provides useful reports about customer order history, product-wise orders and customer purchase analysis.

Orders Table Creatio:

The **Orders** table stores the basic information about customer orders. It contains Order ID, Customer ID, Order Date, Total Amount and Order Status.

Order Details Table Creation:

The **Order Details** table is used to maintain the individual products included in each order. It stores Order Detail ID, Order ID, Product ID, Quantity and Price.

```
SQL> CREATE TABLE ORDERS (Order_ID VARCHAR2(10), Customer_ID NUMBER(38), Order_Date DATE, Total_Amount NUMBER(10,2), Order_Status VARCHAR2(20));
Table created.

SQL> CREATE TABLE ORDER_DETAILS (Order_Detail_ID VARCHAR2(10), Order_ID VARCHAR2(10), Product_ID VARCHAR2(10), Quantity NUMBER, Price NUMBER(10,2));
Table created.
```

Relationship Between Tables:

The Orders and Order Details tables are connected with the Customer and Product tables to maintain proper data relationships.

Customer – Orders Relationship:

A customer can place multiple orders. Therefore, the relationship between Customer and Orders is represented as **one-to-many**.

Orders – Products Relationship:

One order can contain multiple products, and one product can appear in multiple orders. Therefore, the relationship between Orders and Products is represented as **many-to-many**.

This relationship is implemented using the **Order Details** table.

Defining Primary Keys:

Primary keys are added to uniquely identify each record in the Orders and Order Details tables.

- **Order_ID** – Primary Key of Orders table
- **Order_Detail_ID** – Primary Key of Order Details table

Defining Foreign Keys:

Foreign keys are used to establish connections between the Order Details table and the existing Orders and Product tables.

- **Order_ID** references the Orders table.
- **Product_ID** references the Product table.
- **Customer_ID** connects the Orders table with the Customer table.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT FK_Orders_Customer FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID);
Table altered.

SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_OD_Orders FOREIGN KEY (Order_ID) REFERENCES ORDERS(Order_ID);
Table altered.

SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_OD_Product FOREIGN KEY (Product_ID) REFERENCES PRODUCT(Product_ID);
Table altered.
```

NOT NULL Constraint:

The NOT NULL constraint is applied to important fields to ensure that required order information is not left empty.

For example, Order ID, Customer ID, Order Date and Product ID can be maintained as mandatory fields.

```
SQL> ALTER TABLE ORDERS MODIFY Order_Date NOT NULL;  
  
Table altered.
```

Check Constraint:

The CHECK constraint is used to validate values entered into the table.

- Quantity should be greater than 0.
- Price should be greater than or equal to 0.

This helps maintain valid order and product information.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_Total CHECK (Total_Amount >= 0);  
  
Table altered.  
  
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_Status CHECK (Order_Status IN ('Pending', 'Shipped', 'Delivered'));  
  
Table altered.  
  
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_Qty CHECK (Quantity > 0);  
  
Table altered.
```

Adding Order Records:

Order details are inserted into the Orders table using SQL INSERT statements. The records contain Order ID, Customer ID, Order Date, Total Amount and Order Status.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHKO_Status CHECK (Order_Status IN ('Pending', 'Shipped', 'Delivered'));  
  
Table altered.
```

Adding Order Details Records:

Order Details records are created using the products already available in the Product table. Each record contains Order Detail ID, Order ID, Product ID, Quantity and Price.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHKO_Qty CHECK (Quantity > 0);  
Table altered.  
  
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHKO_OD_Price CHECK (Price >= 0);  
Table altered.
```

Customer Order History Report

This report provides information about the customer's previous orders.

The report displays:

- Customer Name
- Order ID
- Order Date
- Total Amount
- Order Status

The report helps to identify the complete order history of each customer.

```

SQL> INSERT INTO ORDERS VALUES ('0301',101,TO_DATE('20-09-2026','DD-MM-YYYY'),1800,'Pending');
1 row created.

SQL> INSERT INTO ORDERS VALUES ('0302',102,TO_DATE('21-09-2026','DD-MM-YYYY'),2500,'Pending');
1 row created.

SQL> INSERT INTO ORDERS VALUES ('0303',103,TO_DATE('22-09-2026','DD-MM-YYYY'),3200,'Pending');
1 row created.

SQL> INSERT INTO ORDERS VALUES ('0304',104,TO_DATE('23-09-2026','DD-MM-YYYY'),4500,'Pending');
1 row created.

SQL> INSERT INTO ORDERS VALUES ('0305',105,TO_DATE('24-09-2026','DD-MM-YYYY'),5200,'Pending');
1 row created.

SQL> INSERT INTO ORDERS VALUES ('0306',101,TO_DATE('24-09-2026','DD-MM-YYYY'),3600,'Pending');
1 row created.

SQL> INSERT INTO ORDERS VALUES ('0307',102,TO_DATE('25-09-2026','DD-MM-YYYY'),6100,'Pending');
1 row created.

SQL> INSERT INTO ORDERS VALUES ('0308',103,TO_DATE('25-09-2026','DD-MM-YYYY'),1600,'Pending');
1 row created.

```

```
SQL> SELECT * FROM ORDERS;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
0301	101	20-SEP-26	1800	Pending
0302	102	21-SEP-26	2500	Pending
0303	103	22-SEP-26	3200	Pending
0304	104	23-SEP-26	4500	Pending
0305	105	24-SEP-26	5200	Pending
0306	101	24-SEP-26	3600	Pending
0307	102	25-SEP-26	6100	Pending
0308	103	25-SEP-26	1600	Pending
0309	104	26-SEP-26	2800	Pending
0310	105	26-SEP-26	7500	Pending

```
10 rows selected.
```

Product-wise Order Report:

This report provides information about how products are ordered by customers.

The report identifies:

- Number of times each product was ordered
- Total quantity sold

This report helps to understand product-wise sales activity

```
SQL> INSERT INTO ORDER_DETAILS VALUES ('OD308','0308','P308',2,800);
```

```
1 row created.
```

```
SQL> INSERT INTO ORDER_DETAILS VALUES ('OD309','0309','P309',2,1400);
```

```
1 row created.
```

```
SQL> INSERT INTO ORDER_DETAILS VALUES ('OD310','0310','P310',3,2500);
```

```
1 row created.
```

```
SQL> INSERT INTO ORDER_DETAILS VALUES ('OD305','0305','P305',4,1300);
```

```
1 row created.
```

```
SQL> INSERT INTO ORDER_DETAILS VALUES ('OD306','0306','P306',2,1800);
```

```
1 row created.
```

```
SQL> select * from order_details;
```

ORDER_DETA	ORDER_ID	PRODUCT_ID	QUANTITY	PRICE
OD301	0301	P301	2	900
OD302	0302	P302	3	850
OD303	0303	P303	1	3200
OD305	0305	P305	4	1300
OD306	0306	P306	2	1800
OD308	0308	P308	2	800
OD309	0309	P309	2	1400
OD310	0310	P310	3	2500

```
8 rows selected.
```

Update order:

```
SQL> UPDATE ORDERS SET TOTAL_AMOUNT = 2500 WHERE ORDER_ID = '0302';

1 row updated.
```

Delete order:

```
SQL> DELETE FROM orders WHERE order_id = '0304';

1 row deleted.
```

Customers with maximum orders:

```
SQL> SELECT customer_name, order_id, order_date, total_amount FROM customer JOIN orders ON customer.customer_id = orders.customer_id;
```

CUSTOMER_NAME	ORDER_ID	ORDER_DAT	TOTAL_AMOUNT
AJAY	0301	20-SEP-26	1800
kavya	0302	21-SEP-26	2500
MANOJ	0303	22-SEP-26	3200

Total quantity:

```
SQL> SELECT p.product_name, SUM(od.quantity) AS total_quantity FROM product p JOIN order_details od ON p.product_id = od.product_id GROUP BY p.product_name;
```

PRODUCT_NAME	TOTAL_QUANTITY
Room 101	2
Room 102	3

Order count:

```
SQL> SELECT c.customer_name, COUNT(o.order_id) AS order_count FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name;
```

CUSTOMER_NAME	ORDER_COUNT
kavya	2
MANOJ	2
NISHA	1

CUSTOMER_NAME	ORDER_COUNT
AJAY	2
RAVI	2

Total spending by each customer:

```
SQL> SELECT c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name;
```

CUSTOMER_NAME	TOTAL_SPENT
kavya	8600
MANOJ	4800
NISHA	2800
AJAY	5400
RAVI	12700

Average order value:

```
SQL> SELECT AVG(total_amount) AS average_order_value FROM orders;
```

AVERAGE_ORDER_VALUE
3811.1111

Conclusion

The **Orders and Order Details Management System** provides an organized method for maintaining customer order and product information. The database uses primary keys, foreign keys and constraints to maintain data consistency.

The order reports help identify customer order history, product-wise order quantities, customers with maximum orders, total customer spending and average order value. Thus, the system provides an effective way to manage and analyse customer orders.